Product Requirements Document (PRD): AI "Panic Button" & Dynamic Budget Rebalancer

Document Version: v1.0
Target Timeline: Month 2 R&D Sprint
Feature Type: Advanced AI / Interactive Frontend Simulation

1. Executive Summary & Objective

Building upon the completed baseline architecture of the Smart Event Planner, this R&D feature introduces an intelligent recovery mechanism for users who face real-world budget overruns. The "Panic Button" transforms static financial logging into an interactive, AI-driven mitigation tool. When an event category goes over budget, the system allows users to execute an automated, cross-category financial rebalancing strategy to maintain their overarching budget cap.

2. Core User Experience & User Flow

1. **The Trigger:** The user enters an actual expense in 3.5 Expense Logging that pushes a specific vendor category (e.g., Catering) into the red (Over Budget).
2. **The Alert:** The 3.6 Budget Tracker dashboard triggers a contextual UI notification chip next to the over-budget category stating: *"Budget overrun detected. Rebalance remaining categories to stay on track?"*
3. **The Playground:** Clicking the chip opens a dedicated modal interface—the **Rebalancer Sandbox**.
4. **The Execution:** The user selects a optimization mode (e.g., Conservative, Aggressive, or AI-Suggested). The system dynamically adjusts the *Allocated Budget* percentages of remaining, unspent categories.
5. **The Preview & Commit:** The frontend animates the changes on a comparison chart. The user can review the adjusted metrics and click **"Commit Changes"** to permanently overwrite their previous budget allocation.

3. Detailed Functional Specifications

3.1 The "Rebalancer Sandbox" Modal Interface

The modal must serve as a non-destructive simulation environment. No database updates are made until the user explicitly commits to the new distribution plan.

UI Element	Functional Behavior	Technical Constraints
Overrun Analytics Banner	Displays exactly how much money ($ or PKR) the user is over budget, alongside the total amount of liquidity left across unspent categories.	Calculated as: Total Deficit = Sum(Actual Cost - Allocated Amount) for all over-budget sectors.
Strategy Toggles	Radio options allowing users to select how the deficit is absorbed: Proportional, Targeted, or AI-Optimized.	Defaults to Proportional (even distribution across flexible categories).
Interactive Comparison Chart	A dual-bar chart or side-by-side pie charts showing current allocation vs. proposed rebalanced allocation.	Must update fluidly on the frontend without initiating full page reloads or API calls.
"Lock" Category Toggles	A list of checkbox or lock icons next to remaining categories, allowing users to exempt specific categories from being reduced.	If a category is locked, its allocated budget remains immutable during recalculation loops.

3.2 Rebalancing Strategies & Mathematical Logic

The intern will need to build frontend and/or lightweight backend logic to distribute the over-budget deficit. Categories with a Payment Status = "Paid" or those manually locked by the user are marked as **Immutable**.

- **Proportional Strategy:** The deficit is split proportionally among all remaining *Mutable* categories based on their current allocation percentage.
- **AI-Optimized Strategy:** The system sends the event type, guest count, and current category spending to an LLM. The AI returns optimized percentages based on historical priority (e.g., it will heavily slash "Decor" and "Entertainment" budgets before touching "Catering" or "Venue").

4. Technical Implementation & Architecture (Month 2 Focus)

4.1 Frontend Interactivity & State Management

- **State Isolation:** The intern should copy the current event's Vendor Category array into a temporary frontend local state (e.g., React/JS Component State or Pinia/Vuex depending on the JS stack). All sandbox tinkering happens strictly within this local array.

- **Animations:** Use a charting library (like Recharts, Chart.js, or ApexCharts) that supports smooth transition animations. When a strategy is changed, the chart elements must visibly animate into their new dimensions.

4.2 Backend & API Extension Requirements

Though the frontend handles user interaction, the existing Laravel backend requires minor additions to support this workflow:
POST /api/events/{event_id}/rebalance-preview
- Input: { locked_category_ids: [], deficit_amount: float, strategy: string }
- Output: Array of proposed category models with updated 'suggested_budget_percentage' and 'allocated_amount'.

POST /api/events/{event_id}/rebalance-commit
- Input: Array of updated categories [{ id: 1, allocated_amount: 500 }, { id: 2, allocated_amount: 350 }]
- Output: Success status, triggers re-calculation of 3.6 Budget Tracker health status.

5. Out of Scope (R&D Constraints)

To ensure successful delivery within the remaining 4 weeks, the following complexities are intentionally excluded:

- **Auto-negotiation suggestions:** The AI will not write emails or suggest specific vendors to negotiate down costs; it only manipulates the internal tracking budget.
- **Multi-currency conversion within sandbox:** The sandbox assumes the baseline event currency (USD or PKR) remains unified and static.
- **Automatic Expense Adjustments:** The system will never alter historical logged *Actual Costs*; it only reduces future *Allocated Budgets*.

6. Acceptance Criteria for Month 2 Deliverable

The R&D feature is considered successfully completed when:

- Logging an expense that exceeds a category's budget successfully fires the "Panic Button" UI banner.
- Opening the sandbox shows accurate, real-time metrics of the overrun amount and flexible remaining cash.
- Changing rebalancing options alters the interactive frontend data visualization charts dynamically with smooth animations.
- Checking a "Lock" icon successfully protects that category's allocation from changing during simulation.
- Clicking "Commit Changes" overwrites the event’s budget profile in the database, updating the progress dashboard instantly without errors.